

Aula 12 - Grafos (Estrutura, DFS, BFS)

Na aula passada, falamos muito sobre a Teoria dos Grafos, isto é, a parte matemática dos grafos. Mas, na prática, estamos interessados na parte de Algoritmos em Grafos.

Representando um grafo

Matematicamente é muito fácil representar um grafo, mas, como fazemos para salvar um grafo na hora de programar um algoritmo? Não conseguimos desenhar bolinhas e tracinhos no VS code, afinal.

Existem algumas formas de fazer isso, vamos vê-las e suas respectivas vantagens.

Lista de arestas

Se armazenarmos apenas as arestas do grafo, temos tudo que precisamos para trabalhar. Afinal, isso praticamente define o grafo inteiro. Então, podemos usar:

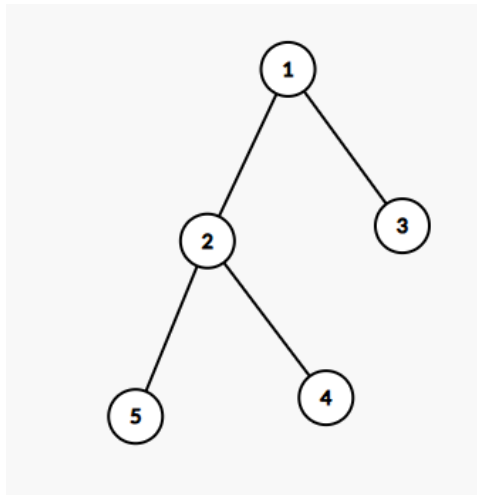
```
vector<pair<int,int>> edges;
```

Essa implementação é extremamente simples e funciona legal para alguns algoritmos, mas tem algumas desvantagens. Para verificar se existe uma aresta entre dois vértices quaisquer que sejam, precisamos percorrer o vetor inteiro. Para achar todos os vizinhos de um vértice, também temos que ver o vetor inteiro.

Matriz de adjacência

Se temos n vértices, declaramos uma matriz de booleanos $n \times n$. Definimos a posição (i, j) da matriz como `true` (ou 1) se existe aresta entre o vértice i e o vértice j , e `false` caso contrário.

	1	2	3	4	5
1	0	1	1	0	0
2	1	0	0	0	1
3	1	0	0	1	0
4	0	0	1	0	0
5	0	1	0	0	0



Essa representação é perfeita para consultar se existe aresta entre u e v , mas ainda não é tão boa para percorrer os vizinhos de um vértice. Ela também é péssima em termos de memória, para grafos grandes, vamos precisar de MUITA memória para representá-lo. Notem que, como o grafo não é direcionado, a matriz é simétrica.

Lista de adjacência

Cada vértice guarda apenas quem são seus vizinhos.

```
vector<int> adj[n]; // muito comum, menos intuitiva
vector<vector<int>> adj(n); // tambem funciona
```

Aqui, conseguimos iterar pelos vizinhos de v apenas iterando pelos elementos de `adj[v]`. Usa pouca memória, e para verificar se u liga em v temos uma operação na ordem $O(d(u))$, até que bom.

Conclusões gerais

Denotando V como o número de vértices e E como o número de arestas, temos:

Estrutura	Memória	Existe aresta?	Percorrer vizinhos
Matriz de adjacência	$O(V^2)$	$O(1)$	$O(V)$
Lista de adjacência	$O(V+E)$	$O(\text{grau})$	$O(\text{grau})$
Lista de arestas	$O(E)$	$O(E)$	$O(E)$

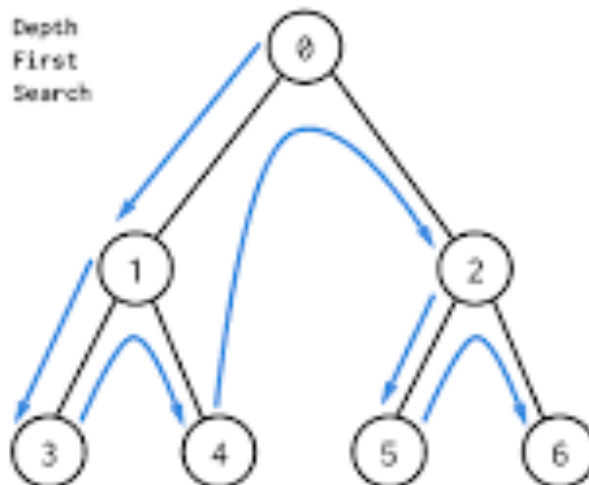
Para os algoritmos dessa aula, a melhor estrutura é a lista de adjacência.

Busca em Profundidade (DFS)

Tanto esse quanto o outro algoritmo que vemos nessa aula têm como objetivo um único: realizar uma busca pelo grafo inteiro, passando por todos os vértices uma única vez. Qual o objetivo da busca? Depende do problema, o algoritmo é genérico e visa apenas fornecer o percurso ideal para que você passe por todos os vértices de forma inteligente.

Na DFS, queremos fazer o seguinte: vamos entrar o mais fundo possível em um grafo, indo sempre para vértices que ainda não visitamos. Quando encontrarmos um beco sem saída (um vértice que não tem mais vizinhos ainda não visitados) vamos voltando pelo mesmo caminho que viemos, até acharmos um vértice que possui um vizinho ainda não visitado.

Essa imagem de resolução cômica que eu achei na internet descreve bem o percurso de uma DFS.



A ideia aqui pode parecer meio contraintuitiva a princípio, é necessário um exercício mental para perceber que de fato ela funciona. Basicamente, ao entrar em um vértice v , olhamos para todos os vizinhos dele que ainda não foram visitados, colocamos todos eles em uma pilha, e o próximo vértice que visitaremos é sempre o que está no topo da pilha. O algoritmo em pseudocódigo fica parecido com:

```
DFS(v) // v é o vértice de início da DFS, no grafo acima, é o vértice 0
    pilha s; // essa pilha representa os vértices que ainda temos que v
    visitar
    pilha.push(v); // e na ordem em que vamos visitá-los
    visitar v;

    enquanto(pilha não estiver vazia)
        u = s.top(); // u é o vértice que visitaremos agora (o do topo
        da pilha)
        s.pop();

        para todo vizinho w de u
            se(w ainda não foi visitado)
                visitar w;
                s.push(w);
```

Experimentem simular esse algoritmo no grafo da resolução cômica acima para tentar entender melhor o funcionamento dessa busca antes de seguir adiante.

Uma vez compreendida essa versão da DFS, podemos também implementá-la de forma recursiva (uma forma bem mais simples de implementar, inclusive, porém eu considero ela ainda mais difícil de entender).

```
DFS_rec(v)
    visitar v;
    para todo vizinho u de v
        se(u ainda não foi visitado)
            dfs_rec(u)
```

Experimentem simular essa versão também. Ela resulta em um percurso com o mesmo comportamento da versão iterativa, porém podem haver algumas variações entre uma e outra a depender do formato do grafo (às vezes elas seguem caminhos meio diferentes). As duas têm o mesmo resultado final todavia.

Entendido o pseudocódigo (parte mais difícil, recomendo passar um bom tempo fazendo o exercício mental de entender o cerne do algoritmo primeiro), fica fácil traduzir para C++. Abaixo eu deixei o código completo das duas versões, usando um grafo implementado com uma lista de adjacências.

```
#include <iostream>
#include <vector>
#include <stack>

using namespace std;

const int MAX = 100005; // número máximo de vértices que suportamos
vector<int> adj[MAX]; // array de vector<int>
bool visitado[MAX]; // array de bools -> visitado[i] representa se i foi
i visitado ou não

void dfs_iterativa(int v) {
    stack<int> s;
    s.push(v);
    visitado[v] = true;

    while (!s.empty()) {
        int u = s.top();
        s.pop();

        for (int w : adj[u]) {
            if (!visitado[w]) {
                visitado[w] = true;
                s.push(w);
            }
        }
    }
}
```

```

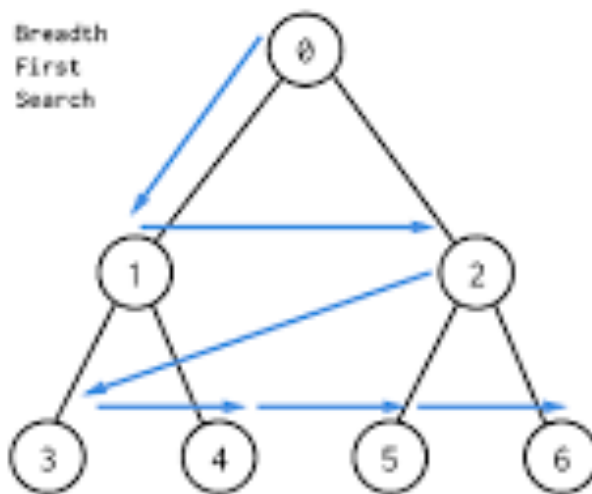
}

void dfs_rec(int v) {
    visitado[v] = true;
    for (int u : adj[v]) {
        if (!visitado[u]) {
            dfs_rec(u);
        }
    }
}
}

```

Busca em Largura (BFS)

Esse algoritmo de busca segue um caminho bem diferente do algoritmo anterior. Ele faz um percurso meio que “em camadas” irradiando a partir do vértice de início, tal qual na imagem abaixo:



Esse algoritmo, embora tenha o mesmo fim que a DFS (visitar o grafo inteiro), tem uma utilidade interessante exclusiva dele: descobrir a distância de um vértice qualquer até o vértice de início da BFS. A implementação dele é literalmente igual à da DFS, porém, substituindo a pilha por uma fila:

```

void bfs(int v) {
    queue<int> q;
    q.push(v);
    visitado[v] = true;

    while (!q.empty()) {
        int u = q.front();
        q.pop();
    }
}

```

```

        for (int w : adj[u]) {
            if (!visitado[w]) {
                visitado[w] = true;
                q.push(w);
            }
        }
    }
}

```

E, diferentemente da DFS, não conseguimos simular o comportamento da BFS com uma recursão 😞

Lidando com entradas em questões de grafos

Normalmente, em questões de grafos, as entradas são dadas da seguinte maneira:

A primeira linha contém dois inteiros N e M , significando, respectivamente, o número de vértices e o número de arestas do grafo.

As M linhas subsequentes contém, cada uma, dois inteiros u e v , representando uma aresta entre o vértice u e o vértice v .

Dessa maneira, precisamos receber essas entradas da seguinte forma (quando usamos uma lista de adjacência para representar o grafo):

```

int main() {
    int n, m;
    if (!(cin >> n >> m)) return 0;

    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u); // se o grafo for direcionado, essa linha
        // não existe
    }

    int componentes = 0;
    for (int i = 1; i <= n; i++) { // observação sobre esse loop abaixo
        if (!visitado[i]) {
            dfs_iterativa(i);
        }
    }
}

```

OBS.: O último loop existe para o caso em que o grafo pode não ser conexo. Se o grafo for conexo, de fato, uma única chamada de DFS ou BFS em um vértice qualquer no grafo

desencadeia uma busca no grafo inteiro, mas, se houverem componentes isoladas uma da outra, cada chamada de DFS só visita todos os vértices da mesma componente daquele vértice. Esse último `for` tá ali pra garantir que o grafo inteiro seja visitado.

Vale ressaltar que ambos os algoritmos dessa aula são da ordem $O(n + m)$, em que n é o número de vértices do grafo e m é o número de arestas do grafo.